```
import pandas as pd
import numpy as np

df = pd.read_csv("DS.csv")
df.head()
```

	Emp_id	Name	Dept.	City	Salary	Performance	Experience	Gender	Age	JoiningYear	Attrition	Education	Work
0	100001	Myra Saha	Marketing	Bangalore	31556.0	Good	20.0	Male	44.0	2011	Yes	B.Sc	\
1	100002	Eshani Hayre	Sales	Noida	82314.0	Excellent	0.0	Male	23.0	2025	No	MBA	H
2	100003	Nitya Bath	Finance	Kolkata	26703.0	Average	18.0	Female	47.0	2012	Yes	M.Tech	\

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 13 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Emp_id       8000 non-null   int64
 1   Name         8000 non-null   object
 2   Dept.        8000 non-null   object
 3   City         7900 non-null   object
 4   Salary       7901 non-null   float64
 5   Performance  7900 non-null   object
 6   Experience   7901 non-null   float64
 7   Gender       8000 non-null   object
 8   Age          7900 non-null   float64
 9   JoiningYear  8000 non-null   int64
 10  Attrition    8000 non-null   object
 11  Education    7900 non-null   object
 12  WorkMode     7900 non-null   object
dtypes: float64(3), int64(2), object(8)
memory usage: 812.6+ KB
```

```
df.isnull().sum()
```

	0
Emp_id	0
Name	0
Dept.	0
City	100
Salary	99
Performance	100
Experience	99
Gender	0
Age	100
JoiningYear	0
Attrition	0
Education	100
WorkMode	100

dtype: int64

```
df['City'].fillna('Unknown', inplace=True)
df['WorkMode'].fillna('Hybrid', inplace=True)
```

```
/tmp/ipykernel_974/4014729515.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df
```

```
  df['City'].fillna('Unknown', inplace=True)
/tmp/ipykernel_974/4014729515.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['WorkMode'].fillna('Hybrid', inplace=True)
```

```
df['Salary'].fillna(df['Salary'].mean(), inplace = True)
df['Age'].fillna(df['Age'].median(), inplace = True)
```

```
/tmp/ipykernel_974/2728884959.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['Salary'].fillna(df['Salary'].mean(), inplace = True)
/tmp/ipykernel_974/2728884959.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['Age'].fillna(df['Age'].median(), inplace = True)
```

```
df['Experience'].fillna(df['Experience'].mode()[0], inplace=True)
```

```
/tmp/ipykernel_974/4211796820.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['Experience'].fillna(df['Experience'].mode()[0], inplace=True)
```

```
df['Education'].unique()
```

```
array(['B.Sc', 'MBA', 'M.Tech', 'BCA', 'M.Sc', 'B.Com', 'B.Tech', 'MCA',
       nan], dtype=object)
```

```
from sklearn.preprocessing import LabelEncoder

education_data = ['B.Sc', 'MBA', 'M.Tech', 'BCA', 'M.Sc', 'B.Com', 'B.Tech', 'MCA']
temp = pd.DataFrame({'Education': education_data})

valid_mask = temp['Education'].notnull()
valid_data = temp.loc[valid_mask, 'Education']

le = LabelEncoder()
encoded_values = le.fit_transform(valid_data)
average_encoded_value = int(round(encoded_values.mean()))
average_label = le.inverse_transform([average_encoded_value])[0]
df['Education'] = df['Education'].fillna(average_label)
```

```
from sklearn.preprocessing import LabelEncoder

education_data = ['B.Sc', 'MBA', 'M.Tech', 'BCA', 'M.Sc', 'B.Com', 'B.Tech', 'MCA']
temp_ed = pd.DataFrame({'Education': education_data})

valid_mask = temp_ed['Education'].notnull()
valid_data = temp_ed.loc[valid_mask, 'Education']

le = LabelEncoder()
encoded_values = le.fit_transform(valid_data)
average_encoded_value = int(round(encoded_values.mean()))
average_label = le.inverse_transform([average_encoded_value])[0]
df['Education'] = df['Education'].fillna(average_label)
```

```
from sklearn.preprocessing import LabelEncoder

education_data = ['B.Sc', 'MBA', 'M.Tech', 'BCA', 'M.Sc', 'B.Com', 'B.Tech', 'MCA']
```

```
temp_pf = pd.DataFrame({'Education': education_data})

valid_mask = temp_pf['Education'].notnull()
valid_data = temp_pf.loc[valid_mask, 'Education']

le = LabelEncoder()
encoded_values = le.fit_transform(valid_data)
average_encoded_value = int(round(encoded_values.mean()))
average_label = le.inverse_transform([average_encoded_value])[0]
df['Performance'] = df['Performance'].fillna(average_label)
```

```
df.isnull().sum()
```

	0
Emp_id	0
Name	0
Dept.	0
City	0
Salary	0
Performance	0
Experience	0
Gender	0
Age	0
JoiningYear	0
Attrition	0
Education	0
WorkMode	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.dtypes
```

	0
Emp_id	int64
Name	object
Dept.	object
City	object
Salary	float64
Performance	object
Experience	float64
Gender	object
Age	float64
JoiningYear	int64
Attrition	object
Education	object
WorkMode	object

dtype: object

```
from google.colab import files

df.to_csv('cleaned_ds.csv', index=False)
files.download('cleaned_ds.csv')
```